

Advanced Algorithms: Exam 1 Review Solutions

Dana Randall Spring 2024

Sarthak Mohanty

Exercise 1

A three-way cut is a set of edges such that removing it from the graph yields three connected components. Modify the min-cut algorithm discussed in class to get a three-way min cut. Compute the probability of success of your design and discuss how to boost it to be $1 - o(1)$.

Hint: Stop looking stuff up, you don't need to! Use the process from class (taking special care near the end.)

Solution

Let $G = (V, E)$ be a multigraph without self loops. For $e = (u, v) \in E$, define the contraction with respect to e (denoted G/e) as in lecture. Now our algorithm is as follows:

Starting from the input graph $G = (V, E)$, repeat the following process until only *four* vertices remain:

1. Choose an edge $e = (u, v)$ uniformly at random from E .
2. Set $G = G/e$.

Once four vertices remain, try all ways of partitioning these four vertices into three nonempty parts, and return the minimum three-way cut among them.

Claim: Let $\delta^*(S)$ be a three-way cut of minimum size of the graph $G = (V, E)$ with $|\delta^*(S)| = k$. Then the probability that the algorithm above returns a minimum three-way cut is at least $\frac{1}{\binom{n}{4}}$.

Proof. Denote the edges we contract in the algorithm as $\{e_1, e_2, \dots, e_{n-4}\}$. The algorithm succeeds if none of the contracted edges are in $\delta^*(S)$.

First note that for every intermediate multigraph, the sum of the degrees for any *pair* of vertices is at least k , otherwise we have a three-way cut of size smaller than k by disconnecting two singular vertices from the rest of the graph. Thus

the sum of the degrees for all pairs together is at least $k \times \binom{|V|}{2}$, so

$$\begin{aligned} \sum_{v \in V} \deg(v) \times (|V| - 1) &\geq k \times \binom{|V|}{2} \\ \Rightarrow \sum_{v \in V} \deg(v) \times (|V| - 1) &\geq \frac{k|V|(|V| - 1)}{2} \\ \Rightarrow \sum_{v \in V} \deg(v) &\geq \frac{k|V|}{2}. \end{aligned}$$

We have shown the sum of all the degrees for every vertex in the graph is at least $\frac{k|V|}{2}$. Hence every intermediate graph with $|V|$ vertices has at least $\frac{k|V|}{4}$ edges.

We can now compute the probability that none of our contracted edges are in $\delta^*(S)$. After j contractions, the resulting multigraph G_j contains $n - j$ vertices, so

$$|E(G_j)| \geq \frac{k(n - j)}{4}.$$

Since there are exactly k edges in $\delta^*(S)$, the probability that the next edge is bad is at most

$$\frac{k}{k(n - j)/4} = \frac{4}{n - j}.$$

Therefore,

$$\begin{aligned} \text{P (success)} &= \text{P} (e_1, \dots, e_{n-4} \notin \delta^*(S)) \\ &= \prod_{j=0}^{n-5} \text{P} (e_{j+1} \notin \delta^*(S) \mid e_1, \dots, e_j \notin \delta^*(S)) \\ &\geq \prod_{j=0}^{n-5} \left(1 - \frac{4}{n - j}\right) \\ &= \frac{n - 4}{n} \times \frac{n - 5}{n - 1} \times \frac{n - 6}{n - 2} \times \dots \times \frac{1}{5} \\ &= \frac{4!}{n(n - 1)(n - 2)(n - 3)} \\ &= \frac{1}{\binom{n}{4}}. \end{aligned}$$

Now suppose no edge of $\delta^*(S)$ was contracted. Then the four remaining vertices still refine the original three parts of the optimal three-way cut. Therefore, among the constant number of three-way partitions of these four vertices, one of them corresponds exactly to $\delta^*(S)$. Since we explicitly try all of them and return the minimum, we return a minimum three-way cut. $\square$

In order to boost the probability of success, we run the algorithm $c(n)\binom{n}{4}$ times, where $c(n) = \omega(1)$. Then the probability that at least one run succeeds is

$$1 - \left(1 - \frac{1}{\binom{n}{4}}\right)^{c(n)\binom{n}{4}} \geq 1 - e^{-c(n)} = 1 - o(1).$$

For example, running the algorithm $\binom{n}{4} \ln n$ times gives success probability at least

$$1 - \frac{1}{n}.$$

Exercise 2

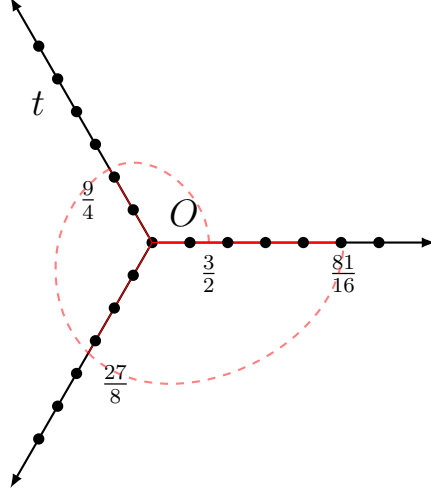
(*Navigating The Labyrinth*) The young maiden Ariadne is at the entrance of a simple labyrinth which branches in $m \geq 2$ corridors, as shown below. Unfortunately, she does not have her magic string with her! She knows the architect of the labyrinth, Daedalus, is somewhere in the labyrinth. She devises the following strategy to find him¹:

Continue until we reach t : execute cycles of steps, where the function determining the number of steps to walk before the i th turn starting from the origin is

$$f(i) = \left(\frac{m}{m-1}\right)^i \quad \forall i \geq 1$$

What is the competitive ratio of this algorithm?

¹This strategy is known as *spiral search* and has been proven to be optimal. Looks like Ariadne was a Theory thread!



Solution

Let $r = \frac{m}{m-1}$. Ariadne searches the m corridors cyclically. If Daedalus is at distance d , the optimal offline cost is $OPT = d$.

Suppose Ariadne finds him on her k -th trip. Her total cost (ALG) includes the out-and-back walks for the first $k - 1$ failed trips, plus the final one-way distance d :

$$ALG = 2 \sum_{i=1}^{k-1} r^i + d$$

To maximize the competitive ratio ALG/OPT , d must be as small as possible. The worst case occurs when Daedalus is located infinitesimally beyond the distance Ariadne reached during her *previous* visit to this exact same corridor. Because she cycles through m corridors, her last visit to this corridor was m steps ago. Thus, $d > r^{k-m}$.

Using the geometric series bound $\sum_{i=1}^{k-1} r^i < \frac{r^k}{r-1}$, we bound the ratio:

$$\frac{ALG}{OPT} = 1 + \frac{2 \sum_{i=1}^{k-1} r^i}{d} < 1 + \frac{2 \left(\frac{r^k}{r-1} \right)}{r^{k-m}} = 1 + \frac{2r^m}{r-1}$$

Substituting $r = \frac{m}{m-1}$ (which implies $r - 1 = \frac{1}{m-1}$), the competitive ratio evaluates to:

$$1 + 2(m-1) \left(\frac{m}{m-1} \right)^m = 1 + \frac{2m^m}{(m-1)^{m-1}}.$$

Exercise 3

(*Potential Functions*) We discussed in class how potential functions were common motifs to help analyze online algorithms.

- (a) For an online algorithm A , let $c_A(\sigma)$ be the cost of the algorithm on the entire sequence and $c_A(i)$ be the cost on request i (and similarly for OPT , the optimal offline algorithm). Prove the following. If for every request sequence σ there exists a potential function Φ such that for all $1 \leq i \leq n$ we have

$$c_A(i) + \Phi(i) - \Phi(i-1) \leq c \cdot c_{\text{OPT}}(i),$$

then A is c -competitive.

- (b) Consider the online problem READ INTO BUFFER: The task is to read an unknown-length non-empty string $s \in \Sigma^+$ of symbols into a buffer stored as contiguous array, one symbol at a time. The buffer must never be larger than $2|s|$ symbols. Reading a symbol has a basic cost of 1. When the currently allocated buffer is full, a new, larger array has to be allocated and the old contents must be copied, incurring a cost of 1 for each symbol already read. In other words, the cost for the k th symbol is either 1 (not full) or k (full).

Devise an online algorithm for this problem and use a potential function to prove that its competitive ratio is at most 3.

Solution

- (a) We assume the usual conditions that $\Phi(0) = 0$ and $\Phi(n) \geq 0$. Then summing over all requests gives

$$\sum_{i=1}^n c_A(i) + \sum_{i=1}^n (\Phi(i) - \Phi(i-1)) \leq c \sum_{i=1}^n c_{\text{OPT}}(i).$$

The potential terms telescope, so

$$c_A(\sigma) + \Phi(n) - \Phi(0) \leq c \cdot c_{\text{OPT}}(\sigma).$$

Since $\Phi(0) = 0$ and $\Phi(n) \geq 0$, we have

$$c_A(\sigma) \leq c_A(\sigma) + \Phi(n) \leq c \cdot c_{\text{OPT}}(\sigma).$$

Therefore A is c -competitive.

- (b) Use the natural doubling algorithm. Start with a buffer of size 1. Whenever the buffer is full, allocate a new buffer of twice the size, copy over the old contents, and then insert the new symbol.

Let n be the number of symbols currently stored and let C be the current capacity. Define the potential function

$$\Phi = 2n - C.$$

After at least one symbol has been read, the doubling algorithm guarantees $n \leq C < 2n$, so $\Phi \geq 0$.

There are two cases.

Case 1: No resize occurs. The actual cost is 1. We increase n by 1, while C stays fixed, so

$$\Delta\Phi = 2.$$

Thus the amortized cost is

$$1 + 2 = 3.$$

Case 2: A resize occurs. Suppose the old buffer is full, so $n = C$. We copy C old symbols and insert the new one, so the actual cost is

$$C + 1.$$

Before resizing,

$$\Phi_{\text{old}} = 2C - C = C.$$

After resizing, the new capacity is $2C$ and the new number of stored symbols is $C + 1$, so

$$\Phi_{\text{new}} = 2(C + 1) - 2C = 2.$$

Therefore

$$\Delta\Phi = 2 - C.$$

So the amortized cost is

$$(C + 1) + (2 - C) = 3.$$

Thus every operation has amortized cost at most 3. Since OPT has to pay at least 1 per symbol, $\text{OPT} \geq |s|$. Therefore the doubling algorithm has competitive ratio at most 3.

Exercise 4

Define a preference list as *cyclic* when it results in a cycle, for instance, where A prefers B over C, B prefers C over A, and C prefers A over B. Assess the accuracy of the following statement: A cyclic preference list leads to the existence of at least one stable marriage.

Solution

The statement is **not accurate as written**.

In the usual stable marriage problem, every person has a strict preference list over the people on the other side. In other words, each preference list is a total ordering. Under this assumption, a stable marriage always exists by the Gale-Shapley algorithm.

So the existence of a stable marriage does *not* come from the preferences being cyclic. It comes from the fact that the preferences are complete strict rankings.

Exercise 5

Consider a scenario in the stable marriage problem involving n men and n women. Determine the *exact* maximum number of proposals that could occur when running the Gale-Shapely algorithm.

Solution

During the Gale-Shapley algorithm, a man never proposes to the same woman twice. Since there are n men and n women, there can be at most n^2 proposals total.

But we can do slightly better. The algorithm stops as soon as every man is matched. Therefore, the only way a man can propose to all n women is if he is the last man to become matched. Every other man must be matched before proposing to all n women, so each of the other $n - 1$ men proposes to at most $n - 1$ women.

Hence the total number of proposals is at most

$$n + (n - 1)(n - 1).$$

Simplifying,

$$\begin{aligned} n + (n - 1)^2 &= n + n^2 - 2n + 1 \\ &= n^2 - n + 1. \end{aligned}$$

Therefore, the number of proposals is at most

$$n^2 - n + 1.$$

Now we show that this bound is achievable.

Let the men be $m_1, m_2, \dots, m_n$ and the women be $w_1, w_2, \dots, w_n$. Give every man the same preference list:

$$w_1 \succ w_2 \succ \dots \succ w_n.$$

Give the women preferences so that each of the first $n - 1$ women keeps rejecting the earlier men in favor of later men. For example,

$$w_i : m_n \succ m_{n-1} \succ \dots \succ m_1.$$

Then the men fight over w_1 , then over w_2 , then over w_3 , and so on. In the worst case, the algorithm forces all but one man to keep moving down their lists until only the last woman remains.

Thus the upper bound is tight, and the exact maximum number of proposals is

$$n^2 - n + 1.$$

Exercise 6

Consider FLUSHWHENFULL algorithm for paging: when a cache miss occurs and the entire cache is full, evict all pages from the cache. What is the competitive ratio of FLUSHWHENFULL?

Solution

Obviously this is not a practical strategy. But, it turns out this has a competitive ratio of k . The proof is same as marking algorithm proof.

Exercise 7

Consider adding the power of limited lookahead to an algorithm for paging. Namely, fix a constant $f \in \mathbb{N}$. Upon receiving the current page p_i , the algorithm also learns $p_{i+1}, p_{i+2}, \dots, p_{i+f}$. Recall that the best achievable competitive ratio for deterministic algorithms without lookahead is k . Can you improve on this bound with lookahead?

Solution

An on-line algorithm does not gain any advantage in the worst case, since any request sequence $\sigma = \sigma_1, \dots, \sigma_p$ can be replaced by the request sequence $\sigma = (\sigma_1)^l, \dots, (\sigma_p)^l$ in which each request is repeated l times, thus, hiding the future requests from the algorithm.

Exercise 8

(Online Coloring) We briefly covered coloring in our homework; now we will cover it in an online context. Our input sequence consists of the vertices $v_1, \dots, v_n$ of an undirected graph $G = (\{v_1, \dots, v_n\}, E)$. Together with each vertex v_i , we are given the list E_i of all edges connecting v_i to previously given vertices $v_1, \dots, v_{i-1}$. Edges from v_i to vertices v_j with $j > i$ are only revealed once vertex v_j is given. The number of edges and vertices in the graph is not known in advance.

When we are given v_i , we have to choose a color $c(v_i) \in \mathbb{N}$ for v_i in such a way that no vertex adjacent to v_i has color $c(v_i)$. As usual, this choice is final; we cannot change the color later on. We want to minimize the number of colors used.

We want to show that there is no deterministic c -competitive algorithm for this problem for any constant c . For any constant number $2 \leq k \in \mathbb{N}$ of colors and any deterministic online algorithm A , devise a strategy for an adversary that satisfies the following requirements.

- The strategy always produces a forest $T_{A,k}$.
- The online algorithm A uses at least k colors on $T_{A,k}$.

Offline, every forest can be colored with 2 colors; therefore, this implies that A 's competitive ratio is at least $k/2$.

Solution

We prove by induction that for any $k \geq 1$, an adversary can construct a forest forcing A to use k colors distributed across k strictly distinct trees.

BASE CASE ($k = 1$): The adversary reveals a single vertex. A assigns it a color, trivially forming a forest with 1 color in 1 tree.

INDUCTIVE STEP: Assume the claim holds for $k - 1$. The adversary applies the $(k - 1)$ -strategy twice to generate two disjoint forests, F_1 and F_2 . Let C_1 and C_2 be the sets of $k - 1$ colors forced in each. By the hypothesis, the colored vertices in both forests lie in distinct trees.

If $C_1 \neq C_2$, then $|C_1 \cup C_2| \geq k$. The adversary stops. Since F_1 and F_2 are disjoint, we can trivially select k distinct colors that lie in separate trees.

If $C_1 = C_2$, the adversary reveals a new vertex x . For each color in C_1 , the adversary connects x to the corresponding vertex in F_1 . Because these vertices belong to disconnected trees in F_1 , routing them through x merges them into a single tree without creating cycles.

Because x borders every color in C_1 , A must assign x a new color $c \notin C_1$. This single new tree containing x (color c), alongside the $k - 1$ unmodified trees in

F_2 (colors $C_2 = C_1$), provides exactly k colors across k disjoint trees.

CONCLUSION: For any k , the adversary can force A to use k colors on a forest. Since forests are 2-colorable offline, A 's competitive ratio is at least $k/2$. Thus, no deterministic online algorithm is c -competitive for any constant c .

Exercise 9²

(*Online Load Balancing*) We discussed load balancing quite a bit in the beginning of the semester. Here we consider a weighted variant. Suppose we receive a sequence of items, where $a_i \in (0, \infty)$. Moreover, we have a fixed number of m bins with unlimited capacity. We have to pack each incoming item into a bin that we have to fix before the next item is given to us. As in load balancing, we want to minimize the total weight W packed into the fullest bin.

We consider the following algorithm GREEDY for this problem. Initially, all bins are empty. On receiving the i th item a_i , GREEDY packs a_i into the emptiest bin; ties are broken arbitrarily. For an example of the algorithm, refer to Figure 1.

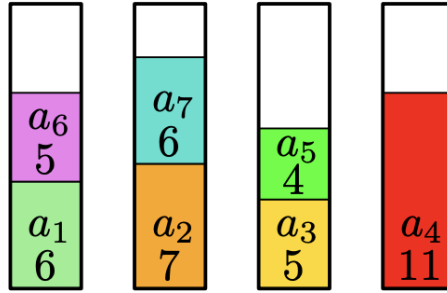


Figure 1: The packing that results from applying GREEDY to $m = 4$ bins and the input sequence $(6, 7, 5, 11, 4, 5, 6)$. The quality of the solution produced by GREEDY is $W_G = 13$ (the fullest bin contains $7 + 6 = 13$ units of weight). The optimal solution has quality $W_O = 11$.

- (1) Prove that GREEDY cannot be better than $(2 - \frac{1}{m})$ -competitive. *Hint: Consider a sequence that begins with small items and ends with a big item.*

²From Technische Universität Braunschweig.

- (2) Prove that GREEDY is $(2 - \frac{1}{m})$ -competitive. *Hint: Consider the last item a_T placed into the fullest bin and the value $W_G - a_T$.*

Solution

Let W_G be the maximum load produced by GREEDY, and W_O be the optimal maximum load.

- (1) Consider an input sequence of $m(m-1)$ items of weight 1, followed by a single item of weight m .

GREEDY perfectly balances the first $m(m-1)$ items, placing exactly $m-1$ items into each of the m bins. The final item of weight m is placed into any bin, yielding:

$$W_G = (m-1) + m = 2m - 1$$

The optimal solution isolates the weight- m item in its own bin and distributes the $m(m-1)$ small items equally among the remaining $m-1$ bins (each also receiving a total load of m). Thus:

$$W_O = m$$

This gives a competitive ratio of:

$$\frac{W_G}{W_O} = \frac{2m-1}{m} = 2 - \frac{1}{m}$$

- (2) Let a_T be the final item placed into the bin that ultimately reaches the maximum load W_G .

Because GREEDY assigns items to the emptiest bin, every other bin must have had a load of at least $W_G - a_T$ immediately prior to a_T 's placement. Therefore, the total weight of all items is at least $m(W_G - a_T) + a_T$.

The optimal solution distributes this same total weight across m bins, meaning its maximum load must be at least the average:

$$W_O \geq \frac{m(W_G - a_T) + a_T}{m} = W_G - a_T + \frac{a_T}{m}$$

Rearranging this inequality isolates W_G :

$$W_G \leq W_O + \left(1 - \frac{1}{m}\right) a_T$$

Since the optimal packing must accommodate a_T in *some* bin, it must be true that $W_O \geq a_T$. Substituting this bounds the ratio:

$$W_G \leq W_O + \left(1 - \frac{1}{m}\right) W_O = \left(2 - \frac{1}{m}\right) W_O$$

Thus, GREEDY is at most $(2 - \frac{1}{m})$ -competitive.

Exercise 10³

(*k-wise Independent Hashing*) We can extend our definition of *k-wise independence* to hashing as well. A hash family $\mathcal{H}$ is *k-wise independent* if for all k distinct keys $x_1, x_2, \dots, x_k \in \mathcal{U}$ and every set of k values $v_1, v_2, \dots, v_k \in \{0, 1, \dots, m-1\}$, we have that

$$P_{h \in \mathcal{H}} (h(x_1) = v_1 \wedge h(x_2) = v_2 \wedge \dots \wedge h(x_k) = v_k) = \frac{1}{m^k}$$

Intuitively, this means that if you look at only up to k keys, the hash family appears to hash them truly randomly.

- (a) Is this hash family from $U = \{a, b\}$ to $\{0, 1\}$ (i.e., $m = 2$) one-wise independent (i.e., uniform)? how about pairwise independent?

	a	b
h_1	0	0
h_2	1	0

- (b) Can you fill in the blanks in this hash family with values in $\{0, 1\}$ to make it pairwise independent? 3-wise independent?

³Taken from CMU's 15-451/651 recitation.

	a	b	c
h_1	0	0	
h_2			1
h_3	0		
h_4	1	1	0

Solution

- (a) i. One-wise independence implies that for each key, each hash value in the range $\{0, 1\}$ must occur with equal probability $\frac{1}{2}$. But for key b , we have $P(h(b) = 0) = 1$ (since both $h_1(b)$ and $h_2(b)$ are 0), so $\mathcal{H}$ is not one-wise independent.
- ii. Pairwise independence requires that for any two distinct keys, the probability of them mapping to any two hash values is $\frac{1}{4}$. But for keys a, b , we have $P(h(a) = 0 \wedge h(b) = 1) = 0$, so $\mathcal{H}$ is not pairwise independent.
- (b) i. For pairwise independence, each pair of distinct keys should map to any pair of values with equal probability $\frac{1}{4}$. One possible solution is as follows:

	a	b	c
h_1	0	0	0
h_2	1	0	1
h_3	0	1	1
h_4	1	1	0

- ii. For 3-wise independence, each trio of distinct keys should map to any trio of values with equal probability $\frac{1}{8}$. Achieving 3-wise independence requires 8 distinct hash functions for a range of $\{0, 1\}$. With only 4 hash functions, we cannot achieve 3-wise independence for this hash family.

Exercise 11

Here we consider a classic 2-universal hash function. Consider a universe where the keys are k bits long, so $U = 0, \dots, 2^k - 1$. For a matrix $\mathbf{A} \in \{0, 1\}^{n \times k}$, define the function $h_{\mathbf{A}}(x) = \mathbf{A}\mathbf{x}$, where we do addition mod 2. Now consider the family of functions $\mathcal{H} = \{h_{\mathbf{A}} : \mathbf{A} \in \{0, 1\}^{k \times n}\}$

- (a) Prove this function is 2-universal.
- (b) How many random bits does it take to sample a random hash function from this family?
- (c) (*Optional*) What is the running time of h ? How does it compare to, say, a binary search tree?

Solution

- (a) We prove that $\mathcal{H}$ is 2-universal. Let $x \neq y$. Then $x - y \neq 0$ over $\mathbb{F}_2$, so equivalently $x + y \neq 0$. We want

$$h_{\mathbf{A}}(x) \neq h_{\mathbf{A}}(y).$$

This is equivalent to

$$\mathbf{A}x \neq \mathbf{A}y,$$

so

$$\mathbf{A}(x + y) \neq 0.$$

Let $z = x + y$. Since $z \neq 0$, there is some coordinate j such that $z_j = 1$. For each row $\mathbf{a}_i$ of $\mathbf{A}$, the value $\mathbf{a}_i \cdot z$ is equally likely to be 0 or 1, because once all entries except a_{ij} are fixed, the value of a_{ij} uniquely determines the parity. Therefore,

$$\mathrm{P}(\mathbf{a}_i \cdot z = 0) = \frac{1}{2}.$$

The rows are independent, so

$$\mathrm{P}(\mathbf{A}z = 0) = \left(\frac{1}{2}\right)^n = \frac{1}{2^n}.$$

Therefore, for all $x \neq y$,

$$\mathbb{P}(h_{\mathbf{A}}(x) = h_{\mathbf{A}}(y)) = \frac{1}{2^n},$$

so $\mathcal{H}$ is 2-universal.

- (b) To sample a random hash function, we need to sample every entry of $\mathbf{A}$. Since $\mathbf{A}$ has nk entries, this takes nk random bits.
- (c) The running time of $h_{\mathbf{A}}(x)$ is $\mathcal{O}(nk)$, since we compute n dot products, each over k bits. A binary search tree takes $\mathcal{O}(\log N)$ comparisons, where N is the number of stored keys, and each comparison of k -bit keys costs $\mathcal{O}(k)$. So BST lookup takes $\mathcal{O}(k \log N)$.

Hashing is usually better for lookup after the hash is computed, since hash tables give expected $\mathcal{O}(1)$ lookup, while BSTs give $\mathcal{O}(\log N)$ lookup.

Exercise 12

Construct an instance of the stable matching problem with 4 men and 4 women and a stable matching in which every man is matched to his least preferred woman.

Solution

Let the men be

$$m_1, m_2, m_3, m_4$$

and the women be

$$w_1, w_2, w_3, w_4.$$

We want the stable matching

$$M = \{(m_1, w_1), (m_2, w_2), (m_3, w_3), (m_4, w_4)\},$$

where every man is matched to his least preferred woman.

Give the men the following preferences:

$$m_1 : w_2 \succ w_3 \succ w_4 \succ w_1,$$

$$m_2 : w_1 \succ w_3 \succ w_4 \succ w_2,$$

$$m_3 : w_1 \succ w_2 \succ w_4 \succ w_3,$$

$$m_4 : w_1 \succ w_2 \succ w_3 \succ w_4.$$

Give each woman her matched man as her favorite:

$$w_1 : m_1 \succ m_2 \succ m_3 \succ m_4,$$

$$w_2 : m_2 \succ m_1 \succ m_3 \succ m_4,$$

$$w_3 : m_3 \succ m_1 \succ m_2 \succ m_4,$$

$$w_4 : m_4 \succ m_1 \succ m_2 \succ m_3.$$

Then every man is matched to his least preferred woman in M . Also, M is stable: although every man would rather switch to some other woman, every woman is already matched to her favorite man. Therefore, no woman would agree to deviate, so there is no blocking pair.

Exercise 13

Prove that in the output of the Traditional Dating Algorithm (when men propose) at most one man gets his last choice.

Solution

We prove that in the men-proposing Traditional Dating Algorithm, at most one man gets his last choice.

Suppose some man m proposes to his last choice. Then before this proposal, he must have already proposed to every other woman and been rejected. Since women only reject a man when they are holding a proposal from someone they prefer more, each of those $n - 1$ women must currently be engaged.

So right before m proposes to his last choice, $n - 1$ women are engaged, and m is the only unengaged man. Therefore, his last-choice woman is the only unengaged woman.

When m proposes to her, she accepts, and now every man and every woman is engaged. Thus the algorithm terminates immediately.

Hence no second man can later propose to his last choice. Therefore, at most one man gets his last choice.

Exercise 14

Recall that the woman optimal matching is the output of the TDA algorithm when women propose to men. Prove that an instance of the stable matching problem has exactly one stable matching if and only if the man optimal matching is equal to the woman optimal matching.

Solution

Let M be the man-optimal matching, and let W be the woman-optimal matching.

First suppose the instance has exactly one stable matching. Since both M and W are stable matchings, they must be the same matching. Therefore, $M = W$.

Conversely, suppose $M = W$. We prove that there is exactly one stable matching.

Let S be any stable matching. Since M is man-optimal, every man weakly prefers his partner in M to his partner in S . Since W is woman-optimal, every woman weakly prefers her partner in W to her partner in S .

But $M = W$, so call this matching A . Then every man weakly prefers A to S , and every woman weakly prefers A to S .

If $S \neq A$, then at least one man is strictly worse off in S than in A . But then the woman matched to him in A cannot also be weakly worse off in S without creating a contradiction in the matching. Equivalently, the standard optimality theorem says every stable matching lies between the man-optimal and woman-optimal matchings. Since these two endpoints are equal, there is no room for any other stable matching.

Therefore, $S = A$, and the instance has exactly one stable matching if and only if $M = W$.